

GUÍA DE REFACTORIZACIÓN

Java + IntelliJ IDEA

Guía completa para el examen

1. Cómo Analizar un Proyecto

Antes de tocar una sola línea de código, realiza este análisis sistemático:

1.1 Preguntas obligatorias por clase

Para cada clase del proyecto hazte estas tres preguntas:

- **¿Qué hace esta clase?** Descríbelo en una sola frase. Si necesitas 'y' para describirla, tiene demasiadas responsabilidades.
- **¿De qué depende?** Qué otras clases necesita para funcionar. Más dependencias = más riesgo al refactorizar.
- **¿Qué se rompería si la cambio?** Qué partes del sistema usan esta clase.

1.2 Checklist de análisis

Recorre cada clase y apunta lo que encuentres:

1. **Leer Main primero:** Entiende el flujo general de la aplicación.
2. **Listar todas las clases:** Dibuja un mapa mental con sus relaciones.
3. **Por cada clase anotar:** atributos y tipos, métodos y lo que hacen, dependencias.
4. **Buscar código muerto:** Métodos con 'no usages' en IntelliJ (aparecen en gris).
5. **Buscar comentarios:** Especialmente TODO y comentarios que explican lo obvio.
6. **Medir longitud de métodos:** Si un método no cabe en pantalla, es demasiado largo.
7. **Contar niveles de anidamiento:** Más de 2 niveles de if/for es una señal de alerta.

💡 Regla práctica: un método bien hecho debería hacer UNA sola cosa y tener un nombre que la describa completamente sin usar 'y'.

2. Detectar y Clasificar Code Smells

2.1 Tabla completa de smells

Smell	Cómo detectarlo
Nombres poco descriptivos	Variables de 1 letra (n, x, pb), métodos como doStuff(), datos, temp
Comentarios redundantes	Comentarios que repiten lo que el código ya dice ('// incrementamos i')
Import muerto	Imports en gris en IntelliJ, no se usan en ningún sitio del archivo
Código muerto	Métodos con 'no usages', variables declaradas y nunca leídas
Magic Numbers	Números literales (0.9, 7, 3600) sin constante con nombre descriptivo
Long Method	Métodos de más de 15-20 líneas. Si hace varias cosas, hay que dividirlo
Arrow Code / Deep Nesting	Más de 2 niveles de if/for anidados. Aspecto en escalera hacia la derecha
Código duplicado	El mismo bloque de código en dos o más sitios
God Class	Una clase que gestiona demasiadas cosas (Hotel que hace todo)
Primitive Obsession	Usar String/int donde debería haber una clase (tipo habitación como String)
Encapsulación rota	Atributos public en clases de modelo. Acceso directo sin getters/setters
Tipo de retorno incorrecto	Métodos que devuelven boolean pero siempre devuelven true o lanzan excepción
Single Responsibility	Una clase tiene métodos de validación + lógica + persistencia juntos
TODO pendiente	Comentarios TODO que indican código incompleto o incorrecto
Constructor incompleto	Constructor que no inicializa todos los atributos, deja nulls

2.2 Orden de detección recomendado

Sigue este orden para no perderte nada:

8. **Imports grises:** Lo primero y más rápido. Elimínalos.
9. **Atributos public:** Escanea todos los atributos de las clases de modelo.
10. **Comentarios:** Busca comentarios TODO y comentarios que explican lo obvio.
11. **Nombres:** Variables de una letra, métodos con nombres vagos.
12. **Magic Numbers:** Números literales en condiciones y cálculos.
13. **Métodos largos:** Mide visualmente. Si no cabe en pantalla, es largo.
14. **Anidamiento:** Cuenta niveles de llaves. Más de 2 es arrow code.
15. **Código duplicado:** Busca bloques similares, especialmente en switch/if.
16. **God Class:** Cuenta las responsabilidades. Hotel gestionando clientes + habitaciones + reservas = God Class.
17. **Código muerto:** Métodos que no se llaman desde ningún sitio.

3. Cómo Refactorizar Cada Smell

⚠ REGLA DE ORO: Si tienes tests, córrelos después de CADA cambio. Si no tienes tests, haz cambios pequeños y verifica manualmente.

3.1 Eliminar Import muerto

Con IntelliJ:

18. Coloca el cursor sobre el import en gris.
19. Pulsa Alt + Enter.
20. Selecciona 'Remove unused import'.

Manual:

21. Borra la línea del import directamente.

3.2 Eliminar Comentarios redundantes

Con o sin IntelliJ (manual):

22. Identifica comentarios que explican lo obvio (`// incrementamos i`, `// creamos el objeto`).
23. Borra la línea entera del comentario.
24. Conserva solo comentarios que explican el PORQUÉ, no el QUÉ.

💡 Un comentario útil: `// Usamos getDayOfYear() aquí porque la API no permite otro método`. Un comentario inútil: `// calculamos el precio`.

3.3 Eliminar Código muerto

Con IntelliJ:

25. IntelliJ muestra 'no usages' en gris junto al método.
26. Haz clic derecho sobre el método → 'Safe Delete'.
27. IntelliJ verificará que nadie lo usa y lo eliminará.

Manual:

28. Verifica manualmente con Ctrl+F que nadie llama al método.
29. Elimina el bloque completo del método.

3.4 Renombrar (Nombres poco descriptivos)

Con IntelliJ:

30. Coloca el cursor sobre la variable o método a renombrar.
31. En el menú superior ve a Refactor → Rename.
32. Escribe el nuevo nombre descriptivo.
33. Pulsa Refactor. IntelliJ actualiza TODAS las referencias automáticamente.

Manual:

34. Busca con Ctrl+F el nombre actual.
35. Usa Ctrl+R para reemplazar uno a uno, verificando cada cambio.

⚠ Nunca uses Reemplazar Todo sin verificar. Puedes renombrar cosas que no quieres.

3.5 Extraer Constante (Magic Numbers)

Con IntelliJ:

36. Selecciona el número literal (ej: 0.9).
37. Ve a Refactor → Extract → Constant.
38. Escribe el nombre de la constante (ej: DESCUENTO_VIP).
39. Pulsa Refactor. IntelliJ sustituye todas las ocurrencias.

Manual:

40. Añade al principio de la clase: `private static final double DESCUENTO_VIP = 0.9;`
41. Busca con Ctrl+F el número 0.9 en el archivo.
42. Reemplaza cada ocurrencia por `DESCUENTO_VIP`.

Resultado esperado:

```
private static final double DESCUENTO_VIP = 0.9;
private static final double DESCUENTO_ESTANCIA = 0.95;
private static final int DIAS_MINIMOS = 7;
```

3.6 Extraer Método (Long Method)

Con IntelliJ:

43. Selecciona el bloque de código que quieres extraer.
44. Ve a Refactor → Extract → Method.
45. Escribe el nombre del nuevo método.
46. Pulsa Refactor. IntelliJ crea el método y reemplaza el bloque seleccionado por la llamada.

Manual:

47. Identifica el bloque con responsabilidad propia.
48. Crea un método nuevo privado con nombre descriptivo.
49. Mueve el bloque al nuevo método.
50. Identifica qué variables necesita el método (serán parámetros) y qué devuelve.
51. Reemplaza el bloque original por la llamada al nuevo método.

💡 Truco: si puedes describir el bloque en una frase corta, ese es el nombre del método.

3.7 Inline Variable (Variable innecesaria)

Con IntelliJ:

52. Coloca el cursor sobre la variable que se declara y se devuelve inmediatamente.

53. Ve a Refactor → Inline Variable.
54. Selecciona 'Inline all usages, remove the variable'.
55. Pulsa Refactor.

Manual:

56. Sustituye la variable por su valor directamente en el return.

Antes:

```
int resultado = a + b;  
return resultado;
```

Después:

```
return a + b;
```

3.8 Early Return (Arrow Code / Deep Nesting)

Con IntelliJ:

57. Coloca el cursor sobre el if más externo.
58. Pulsa Alt + Enter.
59. Selecciona 'Invert if condition'.
60. Elimina el else resultante manualmente si queda.
61. Repite para cada nivel de anidamiento.

Manual:

62. Identifica el if más externo del método.
63. Invierte la condición (cambia isEmpty() por !isEmpty(), etc.).
64. Mueve el contenido del else antes del if, con return/throw.
65. Elimina el else.
66. Repite para el siguiente nivel.

Antes (Arrow Code):

```
if (!lista.isEmpty()) {  
    if (cliente != null) {  
        if (fechaValida) {  
            // lógica principal  
        }  
    }  
}
```

Después (Early Return):

```
if (lista.isEmpty()) return -4;  
if (cliente == null) return -3;  
if (!fechaValida) return -2;  
// lógica principal
```

3.9 Eliminar Código duplicado

Con IntelliJ:

67. Selecciona el bloque duplicado.

68. Ve a Refactor → Extract → Method.
69. IntelliJ detectará automáticamente otros bloques similares y preguntará si reemplazarlos también.
70. Acepta y confirma.

Manual:

71. Identifica los bloques duplicados.
72. Crea un método con los parámetros que varían entre las copias.
73. Reemplaza cada bloque duplicado por la llamada al método.

3.10 Encapsular atributos públicos

Con IntelliJ:

74. Haz clic derecho en el nombre de la clase.
75. Selecciona Refactor → Encapsulate Fields.
76. Marca los campos que quieres encapsular.
77. Selecciona visibilidad Private para los campos.
78. Selecciona visibilidad Public para los accessors.
79. Desmarca 'Set access' si no necesitas setters.
80. Pulsa Refactor.

Manual:

81. Cambia public a private en cada atributo.
82. Crea un getter para cada atributo:

```
public String getNombre() { return nombre; }
```
83. Corrige los errores de compilación en las clases que accedían directamente.

3.11 Cambiar tipo de retorno incorrecto

Manual (no hay herramienta automática):

84. Cambia la firma del método de boolean a void.
85. Elimina los return true; del método.
86. Corrige los if (validar()) que llaman al método eliminando el if.

Antes:

```
public boolean validarNombre(String nombre) {  
    if (nombre == null) throw new IllegalArgumentException(...);  
    return true;  
}
```

Después:

```
public void validarNombre(String nombre) {  
    if (nombre == null) throw new IllegalArgumentException(...);  
}
```

3.12 Extraer clase (God Class / Single Responsibility)

Con IntelliJ (Extract Delegate):

87. Haz clic derecho en la clase God Class.

88. Selecciona Refactor → Extract Delegate.
89. Escribe el nombre de la nueva clase.
90. Marca los métodos y atributos que pertenecen a la nueva clase.
91. Selecciona visibilidad Public.
92. Pulsa Refactor.

Con IntelliJ (Move Members):

93. Selecciona los métodos estáticos que quieres mover.
94. Ve a Refactor → Move.
95. Escribe el nombre de la clase destino.
96. Pulsa Refactor.

Manual:

97. Crea una clase nueva con el nombre apropiado.
98. Mueve los métodos y atributos correspondientes.
99. En la clase original, crea un atributo del tipo de la nueva clase.
100. Actualiza los métodos de la clase original para delegar en la nueva.

3.13 Extraer interfaz

Con IntelliJ:

101. Haz clic derecho en la clase.
102. Selecciona Refactor → Extract Interface.
103. Escribe el nombre de la interfaz.
104. Marca los métodos que quieres en la interfaz.
105. Pulsa Refactor.

Manual:

106. Crea el archivo de la interfaz.
107. Declara los métodos que quieres (sin cuerpo).
108. En la clase, escribe 'implements NombreInterfaz'.
109. Asegúrate de que los métodos tienen exactamente la misma firma.

4. Crear Jerarquías de Clases

4.1 Clase Abstracta

Usa clase abstracta cuando: varias clases comparten comportamiento común Y tienen comportamiento que varía entre ellas.

Paso a paso — Manual

110. **Crear la clase abstracta:**

```
public abstract class Animal {  
    private String nombre;  
    // Constructor  
    public Animal(String nombre) {  
        this.nombre = nombre;  
    }  
    // Método común (implementado aquí)  
    public String getNombre() { return nombre; }  
    // Método abstracto (cada subclase lo implementa diferente)  
    public abstract String hacerSonido();  
}
```

111. **Crear las clases hijas:**

```
public class Perro extends Animal {  
    public Perro(String nombre) {  
        super(nombre); // Llama al constructor del padre  
    }  
    @Override  
    public String hacerSonido() {  
        return "Guau";  
    }  
}
```

112. **Reglas importantes:**

- La clase abstracta NO puede instanciarse directamente.
- Las subclases DEBEN implementar todos los métodos abstractos.
- El constructor de la subclase DEBE llamar a super() como primera línea.
- Los métodos no abstractos se heredan tal cual.

Paso a paso — Con IntelliJ (Extract Superclass)

113. Haz clic derecho en la clase que va a ser subclase.
114. Selecciona Refactor → Extract Superclass.
115. Escribe el nombre de la clase abstracta.
116. Marca los métodos que quieres subir a la clase padre.
117. Marca 'Make abstract' para los que deben ser abstractos.
118. Pulsa Refactor.

⚠ Si tienes varias subclases, aplica Extract Superclass a la primera y luego haz que las otras hereden manualmente de la clase abstracta creada.

4.2 Interfaz

Usa interfaz cuando: quieres definir un contrato que pueden implementar clases sin relación de herencia.

Paso a paso — Manual

119. **Crear la interfaz:**

```
public interface Validable {  
    void validar(); // Sin cuerpo, sin modificador de acceso  
    boolean esValido(); // Todos los métodos son públicos por defecto  
}
```

120. **Implementar en la clase:**

```
public class Cliente implements Validable {  
    @Override  
    public void validar() {  
        if (nombre == null) throw new IllegalArgumentException(...);  
    }  
    @Override  
    public boolean esValido() { return nombre != null; }  
}
```

121. **Reglas importantes:**

- Una clase puede implementar MÚLTIPLES interfaces (implements A, B, C).
- Una clase solo puede heredar de UNA clase abstracta.
- Todos los métodos de la interfaz son public abstract por defecto.
- Desde Java 8 se pueden tener métodos default con cuerpo en interfaces.

Paso a paso — Con IntelliJ (Extract Interface)

122. Haz clic derecho en la clase que va a implementar la interfaz.
123. Selecciona Refactor → Extract Interface.
124. Escribe el nombre de la interfaz.
125. Marca los métodos que formarán parte de la interfaz.
126. Pulsa Refactor.

4.3 Detalles importantes para el examen

Diferencias clave:

	Clase Abstracta	Interfaz
¿Puede tener constructor?	Sí	No
¿Puede tener atributos?	Sí	Solo static final

¿Puede tener métodos con cuerpo?	Sí (no abstractos)	Solo default (Java 8+)
¿Cuántas puede heredar/implementar?	Solo una (extends)	Múltiples (implements)
¿Cuándo usar?	Clases relacionadas con código común	Contrato entre clases no relacionadas

Patrón Fachada (Facade):

Cuando divides una God Class en gestores, la clase original actúa como Fachada: delega en los gestores sin exponer su implementación interna.

```
public class Hotel {
    private GestorClientes gestorClientes = new GestorClientes();
    private GestorReservas gestorReservas = new GestorReservas();
    // Hotel solo delega, no expone los gestores
    public void registrarCliente(...) { gestorClientes.registrar(...); }
}
```

5. Herramientas de IntelliJ IDEA paso a paso

Todas estas herramientas se encuentran en el menú Refactor de la barra superior, o con clic derecho → Refactor.

5.1 Rename (Renombrar)

Para renombrar variables, métodos, clases o parámetros actualizando todas las referencias.

127. Coloca el cursor sobre el elemento a renombrar.
128. Ve al menú Refactor → Rename.
129. En el popup que aparece, escribe el nuevo nombre.
130. Pulsa Enter para confirmar.

IntelliJ actualiza automáticamente todas las referencias en todo el proyecto.

5.2 Extract Method (Extraer Método)

Para extraer un bloque de código a un método independiente.

131. Selecciona el bloque de código que quieres extraer (clic y arrastra).
132. Ve al menú Refactor → Extract → Method.
133. En el diálogo, escribe el nombre del nuevo método.
134. Revisa los parámetros detectados automáticamente por IntelliJ.
135. Pulsa Refactor.

 IntelliJ detecta automáticamente qué variables necesita el método como parámetros y qué devuelve.

5.3 Extract Constant (Extraer Constante)

Para convertir un valor literal en una constante con nombre.

136. Selecciona el valor literal (número o texto).
137. Ve al menú Refactor → Extract → Constant.
138. Escribe el nombre de la constante (convención: MAYUSCULAS_CON_GUION).
139. Elige si la constante es local o de clase.
140. Pulsa Refactor.

5.4 Inline Variable (Eliminar variable innecesaria)

Para eliminar una variable que solo se declara y se devuelve inmediatamente.

141. Coloca el cursor sobre el nombre de la variable.
142. Ve al menú Refactor → Inline Variable.
143. Selecciona 'Inline all usages, remove the variable'.
144. Pulsa Refactor.

5.5 Encapsulate Fields

Para convertir atributos públicos en privados y generar getters/setters.

145. Haz clic derecho en cualquier parte de la clase.

146. Selecciona Refactor → Encapsulate Fields.
147. En el diálogo, marca todos los campos que quieres encapsular.
148. En 'Encapsulated Fields Visibility' selecciona Private.
149. En 'Accessors Visibility' selecciona Public.
150. Desmarca 'Set access' si no necesitas setters.
151. Pulsa Refactor.

5.6 Move Members (Mover métodos estáticos)

Para mover métodos estáticos de una clase a otra (por ejemplo, al extraer un validador).

152. Coloca el cursor sobre el método estático a mover.
153. Ve al menú Refactor → Move.
154. En el campo 'To (fully qualified name)' escribe el nombre completo de la clase destino.
155. Marca los métodos que quieres mover.
156. Selecciona la visibilidad Public.
157. Pulsa Refactor.

💡 Si la clase destino no existe todavía, escribe su nombre igualmente. IntelliJ la creará.

5.7 Extract Delegate (Extraer Delegado)

Para extraer un conjunto de métodos y atributos a una nueva clase independiente (ideal para God Class).

158. Haz clic derecho en la clase que quieres dividir.
159. Selecciona Refactor → Extract Delegate.
160. Escribe el nombre de la nueva clase.
161. Marca los métodos y atributos que pertenecen a la nueva clase.
162. Selecciona visibilidad Public.
163. Pulsa Refactor.

⚠️ IntelliJ puede generar dependencias circulares si no seleccionas bien los miembros. Revisa el resultado y deshaz con Ctrl+Z si es necesario.

5.8 Extract Superclass (Extraer Clase Padre)

Para crear una clase abstracta a partir de una clase existente.

164. Haz clic derecho en la clase que va a ser subclase.
165. Selecciona Refactor → Extract Superclass.
166. Escribe el nombre de la nueva clase padre.
167. Marca los métodos que quieres mover al padre.
168. Marca 'Make abstract' para los métodos que deben ser abstractos.
169. Pulsa Refactor.

5.9 Extract Interface

Para extraer una interfaz a partir de los métodos de una clase.

170. Haz clic derecho en la clase.

- 171. Selecciona Refactor → Extract Interface.
- 172. Escribe el nombre de la interfaz.
- 173. Marca los métodos que quieres incluir en la interfaz.
- 174. Pulsa Refactor.

5.10 Invert If (para Early Return)

Para invertir la condición de un if y facilitar el patrón Early Return.

- 175. Coloca el cursor sobre la palabra if que quieres invertir.
- 176. Pulsa Alt + Enter para abrir el menú de sugerencias.
- 177. Selecciona 'Invert if condition'.
- 178. Elimina manualmente el else resultante si procede.

5.11 Safe Delete (Borrar con seguridad)

Para eliminar métodos o clases verificando que nadie los usa.

- 179. Haz clic derecho sobre el método o clase.
- 180. Selecciona Refactor → Safe Delete.
- 181. IntelliJ comprueba que no hay referencias. Si las hay, te avisa.
- 182. Confirma la eliminación.

6. Cómo Hacer la Documentación

6.1 Estructura del informe

El informe debe tener esta estructura mínima:

183. **Portada:** Nombre, actividad, proyecto.
184. **Índice:** Con enlaces a cada sección.
185. **Análisis previo:** Un apartado por clase con los smells identificados.
186. **Tests (si los hay):** Capturas de los tests pasando antes de refactorizar.
187. **Refactorizaciones:** Un apartado por cada smell corregido, con captura.
188. **Estructura final:** Tabla de clases y sus responsabilidades.
189. **Resumen de técnicas:** Tabla con las herramientas usadas.

6.2 Cómo hacer las capturas

Para cada refactorización, haz la captura EN EL MOMENTO CLAVE:

- **Rename:** Captura cuando IntelliJ muestra las ocurrencias resaltadas antes de cambiar.
- **Extract Method:** Captura del diálogo antes de pulsar Refactor.
- **Extract Constant:** Captura del popup con el nombre propuesto.
- **Encapsulate Fields:** Captura del diálogo con los campos marcados.
- **Tests en verde:** Captura después de cada grupo de cambios.

6.3 Sintaxis Markdown útil

```
# Título H1
## Subtítulo H2
### Apartado H3
**negrita**
*cursiva*
`código inline`
```java
// bloque de código
```

![texto alternativo](ruta/imagen.png)
| Col1 | Col2 | ← tabla
|-----|-----|
| dato | dato |
```

7. Consejos Adicionales para el Examen

7.1 Orden de trabajo recomendado

190. **Lee todo primero:** Antes de tocar nada, lee todas las clases. 5 minutos de lectura ahorran 20 de depuración.
191. **Anota los smells:** En papel o en un comentario temporal. No intentes arreglar mientras lees.
192. **Lo seguro primero:** Imports muertos, comentarios, renombrar. Son cambios sin riesgo.
193. **Luego los bugs lógicos:** Lógica invertida, tipos de retorno incorrectos.
194. **Por último lo estructural:** Extract Method, Extract Class, jerarquías.
195. **Tests después de cada grupo:** Si tienes tests, córrelos frecuentemente.

7.2 Señales de alarma en el código

- **Método muy largo:** Si hay que hacer scroll para verlo entero, es demasiado largo.
- **Clase con más de 200 líneas:** Probable God Class.
- **Mismo número en varios sitios:** Magic Number.
- **Variable con nombre de una letra:** Nombre poco descriptivo.
- **if dentro de if dentro de if:** Arrow Code, aplicar Early Return.
- **Bloque de código repetido:** Código duplicado, extraer método.
- **Atributos public en clase de modelo:** Encapsulación rota.

7.3 Cómo deshacer en IntelliJ

Si algo sale mal después de una refactorización automática:

- Ctrl + Z: deshace el último cambio.
- Ctrl + Shift + Z: rehace un cambio deshecho.
- IntelliJ mantiene el historial completo de la sesión.

7.4 Verificación final

Antes de entregar, verifica:

- No hay imports en gris.
- No hay métodos con 'no usages'.
- No hay variables de una letra (excepto i, j en bucles).
- No hay números literales en cálculos y condiciones.
- Todos los atributos de clases de modelo son private.
- Los métodos hacen una sola cosa.
- El código se puede leer sin necesitar comentarios que expliquen el qué.

💡 La mejor prueba de que el código está limpio: léelo en voz alta. Si suena natural y entiendes lo que hace sin pensar, está bien refactorizado.

7.5 Atajos de teclado (referencia rápida)

| Acción | Atajo Windows/Linux |
|-------------------------|-----------------------------------|
| Rename | Shift + F6 |
| Extract Method | Ctrl + Alt + M |
| Extract Variable | Ctrl + Alt + V |
| Extract Constant | Ctrl + Alt + C |
| Extract Parameter | Ctrl + Alt + P |
| Inline Variable/Method | Ctrl + Alt + N |
| Move | F6 |
| Encapsulate Fields | Alt + Enter → Encapsulate |
| Invert If | Alt + Enter → Invert if condition |
| Sugerencias / Quick Fix | Alt + Enter |
| Safe Delete | Alt + Delete |
| Formatear código | Ctrl + Alt + L |
| Deshacer | Ctrl + Z |
| Buscar en archivo | Ctrl + F |
| Buscar en proyecto | Ctrl + Shift + F |
| Reemplazar en archivo | Ctrl + R |